Taller 5

Maria Lucía Castillo García

Índice de contenido

Preguntas de reflexión....1
Parte 1: Introducción a Docker....2
Parte 7: Recursos Docker....3
Evidencia de ejecución de cada actividad....5
Parte 2: Explorando Docker....5
Parte 3: Descarga de imágenes....6
Parte 4: Primer contenedor....6
Parte 5: Contenedor web....6
Parte 6: Administración....7
Parte 7: Recursos Docker....11
Reto integrador....11

Preguntas de reflexión

1) ¿Cuál es la diferencia entre una imagen y un contenedor?

- Una imagen contiene todo lo necesario para ejecutar una aplicación, como el sistema operativo, las dependencias y el código. En cambio, un contenedor es una instancia de esa imagen que se encuentra en ejecución.
Es decir, la imagen sería como la plantilla o base, mientras que el contenedor es la aplicación funcionando a partir de esa plantilla. Además, una misma imagen puede generar varios contenedores.

2) ¿Por qué Docker facilita el despliegue en la nube?

- Porque empaqueta la aplicación junto con todas sus dependencias, de manera que puede ejecutarse de la misma forma en distintos entornos. Esto reduce problemas de compatibilidad y hace que sea más sencillo mover aplicaciones desde el computador del desarrollador hasta servidores o servicios en la nube.

3) ¿Qué ventajas ofrece Docker frente a instalar aplicaciones directamente en el sistema operativo?

- Docker permite que cada aplicación tenga su propio entorno aislado, evitando conflictos entre dependencias o versiones de software. Además, facilita la portabilidad, la administración y la replicación de aplicaciones en diferentes equipos.

4) ¿En qué situaciones utilizaría un volumen Docker?

- Utilizaría un volumen cuando necesite conservar datos aunque el contenedor sea eliminado o reiniciado. Por ejemplo, para almacenar bases de datos, archivos de usuarios o configuraciones importantes que no deben perderse.

5) ¿Qué riesgos existen al ejecutar `docker system prune` en un servidor de producción?

- El principal riesgo es eliminar recursos que todavía podrían ser necesarios, como imágenes, contenedores detenidos, redes o cachés. Si no se revisa cuidadosamente qué se va a borrar, se podrían afectar aplicaciones o procesos que dependen de esos recursos.

Parte 1: Introducción a Docker

Concepto	Qué es
Imagen	Es un paquete que contiene una aplicación y todo lo que necesita para funcionar (sistema operativo, librerías, configuraciones, dependencias, código, etc.). No cambia, es solo de lectura.
Contenedor	Es una copia de una imagen que está ejecutándose, es decir, la aplicación ya está corriendo. Como es una instancia en ejecución de una imagen, puede haber varios contenedores de la misma imagen.
Dockerfile	Es un archivo de texto con instrucciones paso a paso para construir una imagen personalizada.
Docker Hub	Es una página donde otras personas publican imágenes para que se puedan descargar y usar.

¿Por qué usar Docker?

Concepto	Significado	¿Por qué Docker?
Portabilidad	La aplicación puede ejecutarse de la misma forma en diferentes computadores, servidores o plataformas en la nube.	Docker incluye todas las dependencias de la aplicación, evitando problemas de compatibilidad entre entornos.
Escalabilidad	Capacidad de aumentar recursos o instancias cuando hay más usuarios o más trabajo.	Permite crear rápidamente varios contenedores de una misma aplicación para distribuir la carga.
Aislamiento	Cada aplicación funciona en su propio entorno sin afectar a las demás.	Un contenedor tiene sus propias dependencias y configuraciones, evitando conflictos con otros programas o con el sistema operativo.
CI/CD	Automatización de pruebas, integración y despliegue de software.	Docker garantiza que la misma imagen probada sea la que se despliega en producción, reduciendo errores.
Microservicios	Arquitectura donde una aplicación se divide en servicios pequeños e independientes.	Cada servicio puede ejecutarse en un contenedor separado, facilitando su mantenimiento, actualización y escalamiento.

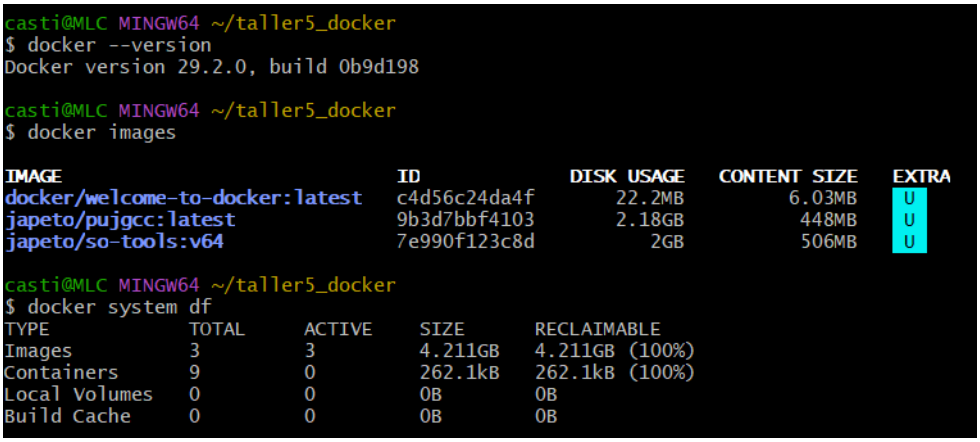
Parte 7: Recursos Docker

Comando	Recurso	¿Qué hace?
`docker network ls`	Redes	Muestra las redes que Docker ha creado para que los contenedores puedan comunicarse entre sí o con el exterior.
`docker volume ls`	Volúmenes	Muestra los volúmenes donde se almacenan datos persistentes que no se pierden al eliminar un contenedor.

Comando	Recurso	¿Qué hace?
`docker system df`	Espacio en disco	Permite ver cuánto espacio ocupan las imágenes, contenedores, volúmenes y cachés.
`docker system prune`	Recursos sin usar	Elimina contenedores detenidos, redes sin uso y otros recursos innecesarios para liberar espacio.

Evidencia de ejecución de cada actividad

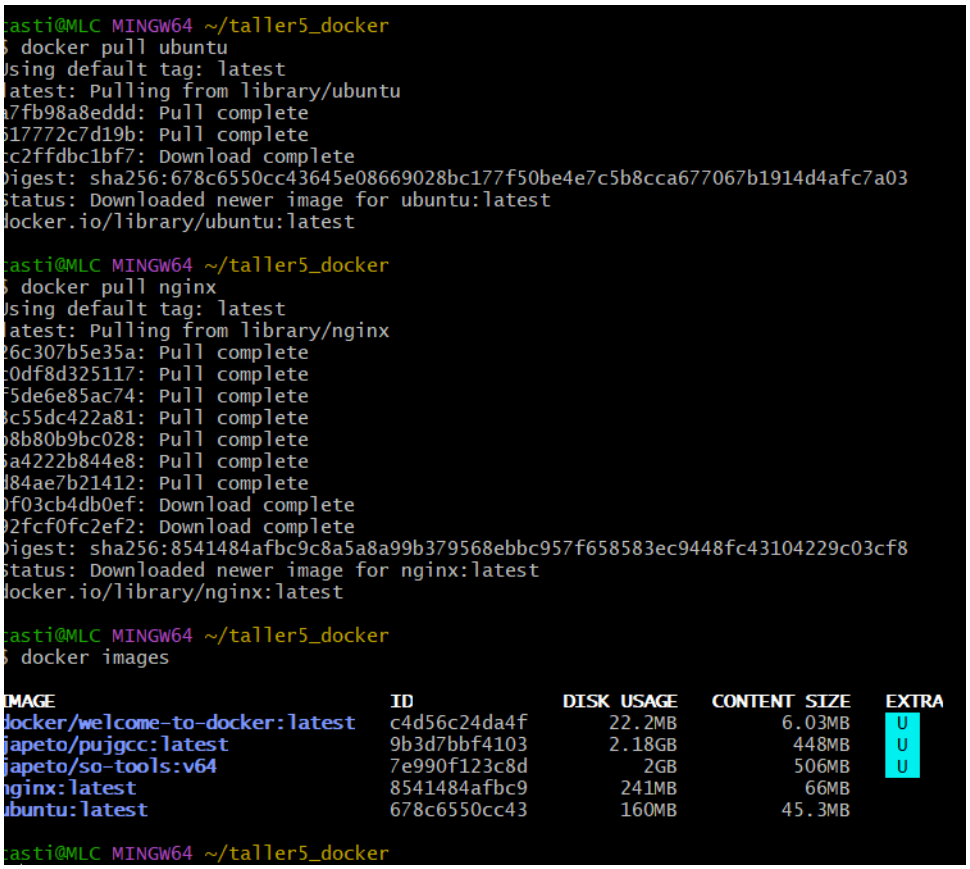
Parte 2: Explorando Docker



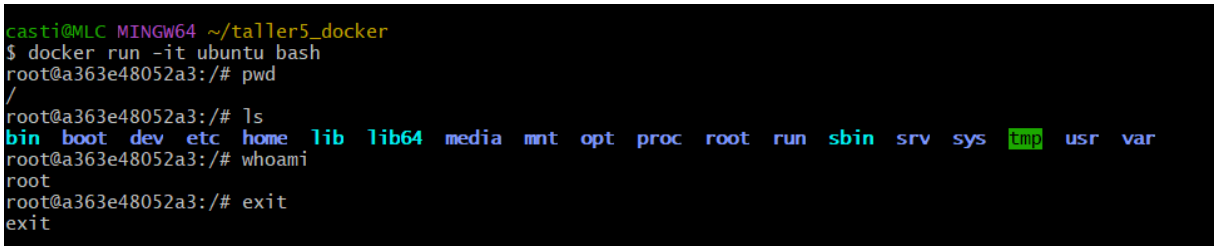
Con `docker images` vemos que ya tenía 3 imágenes descargadas de un proyecto anterior, que en conjunto ocupan cerca de 4.2 GB en disco. Esto tiene sentido con lo que muestra `docker system df`, donde se confirma que esas 3 imágenes están activas (o sea, ninguna quedó "huérfana" sin usarse) y que también hay 9 contenedores creados a partir de ellas, que pesan muy poco porque un contenedor detenido casi no consume espacio extra por sí solo; el peso grande está en las imágenes, no en los contenedores.

Esto confirma la relación clave entre estos dos conceptos: una imagen es la base pesada y reutilizable, mientras que un contenedor es una instancia ligera que se crea a partir de ella.

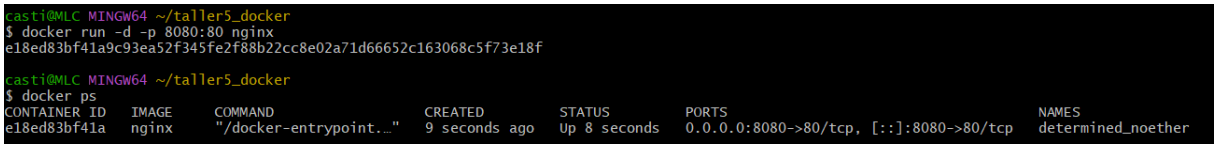
Parte 3: Descarga de imágenes

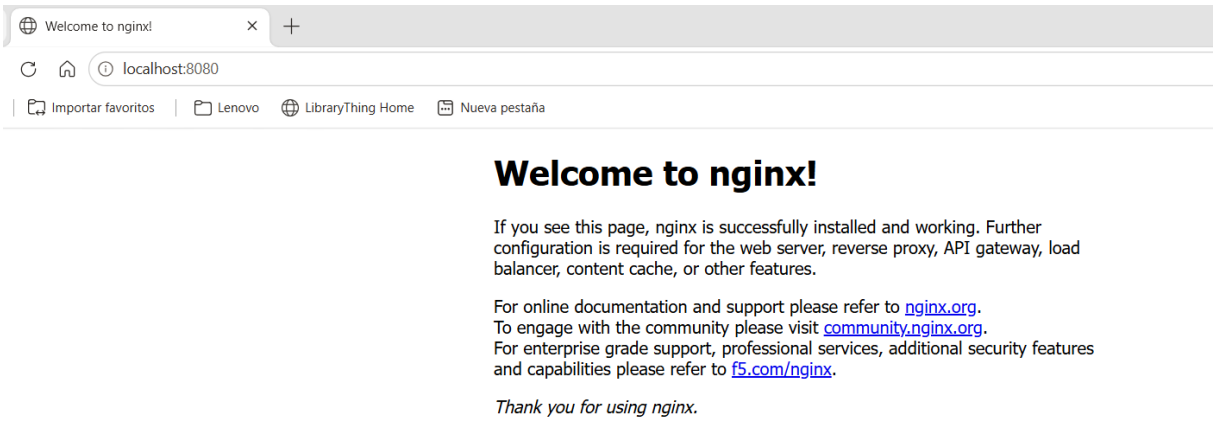


Parte 4: Primer contenedor



Parte 5: Contenedor web

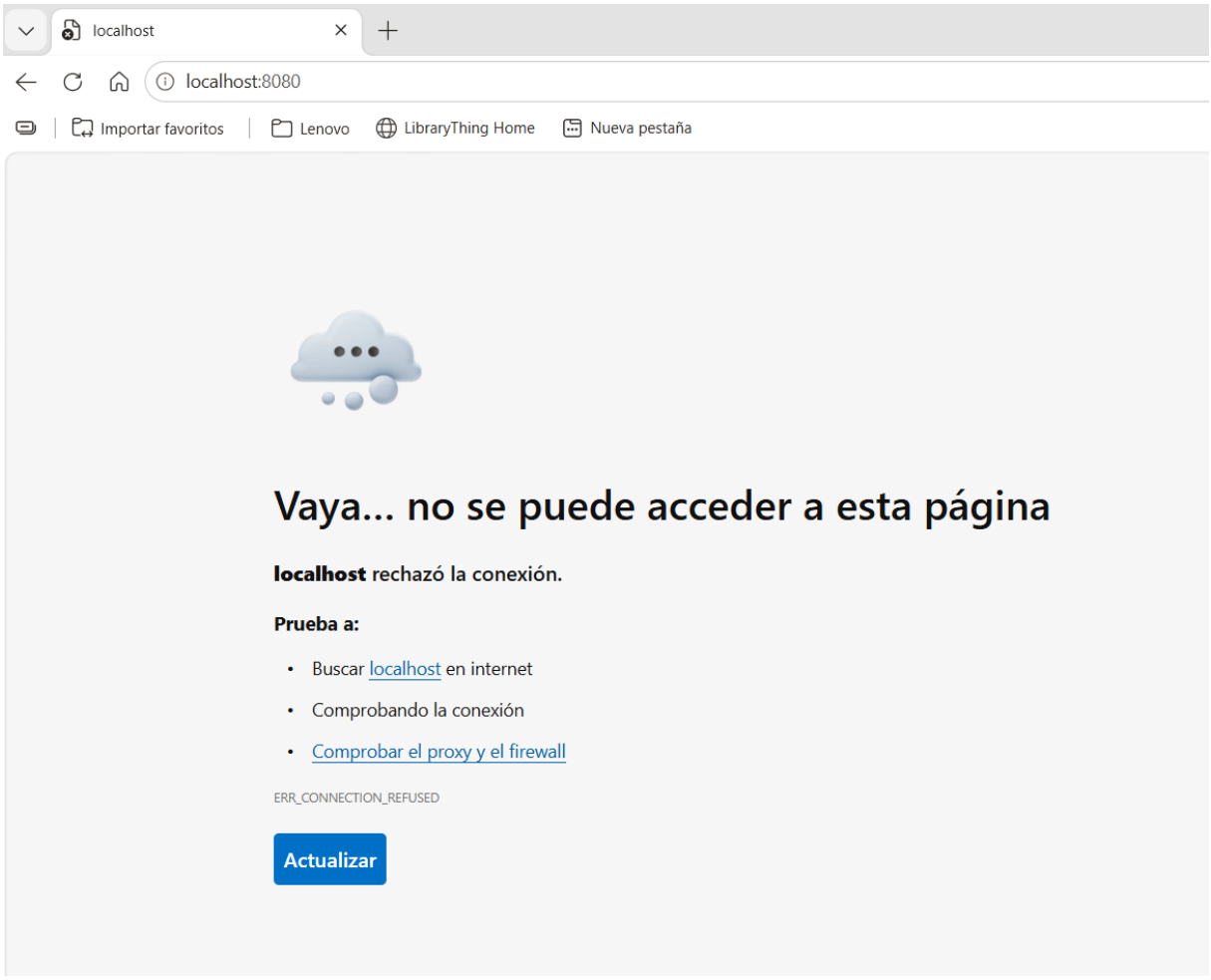




Parte 6: Administración

docker stop

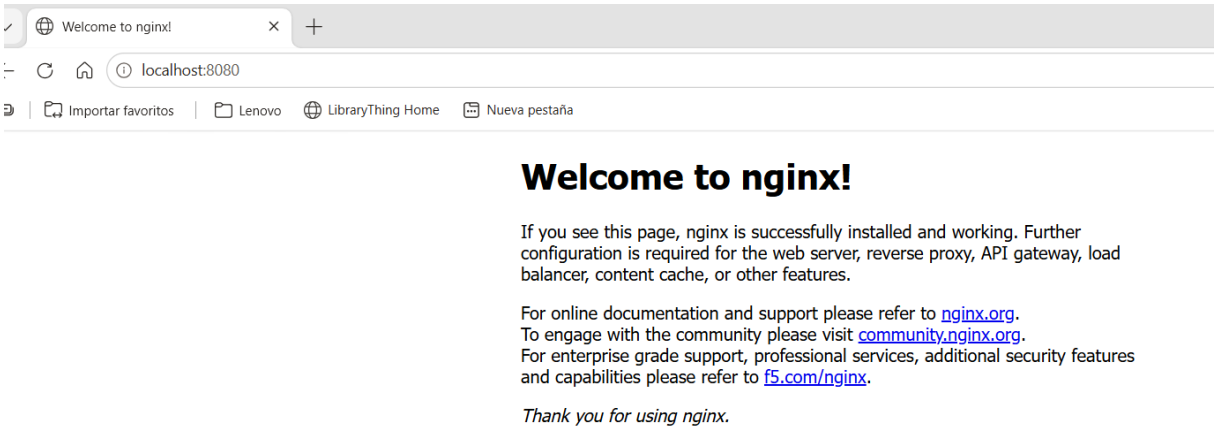
```
casti@MLC MINGW64 ~/taller5_docker
$ docker stop determined_noether
determined_noether
```



Detiene un contenedor que está corriendo. Es como apagar una aplicación.

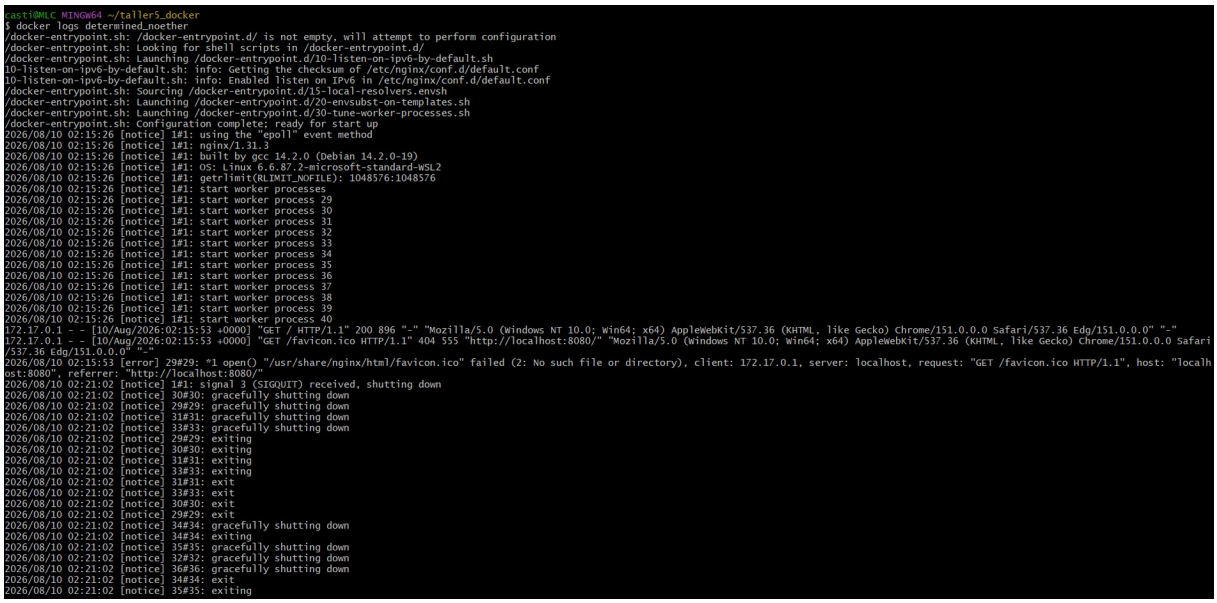
docker start

```
casti@MLC MINGW64 ~/taller5_docker
$ docker start determined_noether
determined_noether
```

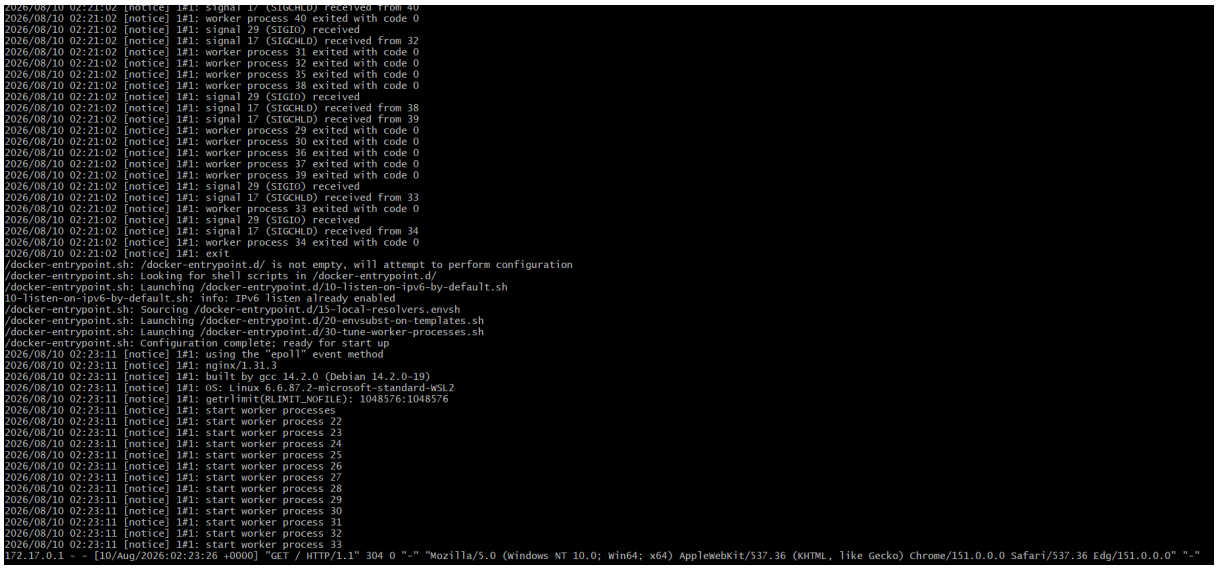


Vuelve a iniciar un contenedor que estaba detenido. Es como volver a encender una aplicación.

docker logs

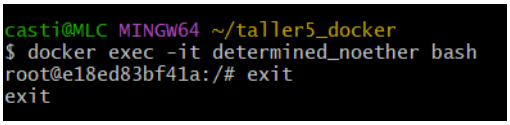


```
casti@MLC MINGW64 ~/taller5_docker
$ docker logs determined_noether
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/10 02:15:26 [notice] 1#1: using the "epoll" event method
2026/08/10 02:15:26 [notice] 1#1: nginx/1.31.3
2026/08/10 02:15:26 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/08/10 02:15:26 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/08/10 02:15:26 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/08/10 02:15:26 [notice] 1#1: start worker processes
2026/08/10 02:15:26 [notice] 1#1: start worker process 29
2026/08/10 02:15:26 [notice] 1#1: start worker process 30
2026/08/10 02:15:26 [notice] 1#1: start worker process 31
2026/08/10 02:15:26 [notice] 1#1: start worker process 32
2026/08/10 02:15:26 [notice] 1#1: start worker process 33
2026/08/10 02:15:26 [notice] 1#1: start worker process 34
2026/08/10 02:15:26 [notice] 1#1: start worker process 35
2026/08/10 02:15:26 [notice] 1#1: start worker process 36
2026/08/10 02:15:26 [notice] 1#1: start worker process 37
2026/08/10 02:15:26 [notice] 1#1: start worker process 38
2026/08/10 02:15:26 [notice] 1#1: start worker process 39
2026/08/10 02:15:26 [notice] 1#1: start worker process 40
172.17.0.1 - - [10/Aug/2026:02:15:53 +0000] "GET / HTTP/1.1" 200 896 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 Edg/151.0.0.0" "-"
172.17.0.1 - - [10/Aug/2026:02:15:53 +0000] "GET /favicon.ico HTTP/1.1" 404 555 "http://localhost:8080/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari
/537.36 Edg/151.0.0.0" "-"
2026/08/10 02:15:53 [error] 29#29: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 172.17.0.1, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "localh
ost:8080", referrer: "http://localhost:8080/"
2026/08/10 02:21:02 [notice] 1#1: signal 3 (SIGQUIT) received, shutting down
2026/08/10 02:21:02 [notice] 30#30: gracefully shutting down
2026/08/10 02:21:02 [notice] 29#29: gracefully shutting down
2026/08/10 02:21:02 [notice] 31#31: gracefully shutting down
2026/08/10 02:21:02 [notice] 33#33: gracefully shutting down
2026/08/10 02:21:02 [notice] 29#29: exiting
2026/08/10 02:21:02 [notice] 30#30: exiting
2026/08/10 02:21:02 [notice] 31#31: exiting
2026/08/10 02:21:02 [notice] 33#33: exiting
2026/08/10 02:21:02 [notice] 31#31: exit
2026/08/10 02:21:02 [notice] 33#33: exit
2026/08/10 02:21:02 [notice] 30#30: exit
2026/08/10 02:21:02 [notice] 29#29: exit
2026/08/10 02:21:02 [notice] 34#34: gracefully shutting down
2026/08/10 02:21:02 [notice] 34#34: exiting
2026/08/10 02:21:02 [notice] 35#35: gracefully shutting down
2026/08/10 02:21:02 [notice] 32#32: gracefully shutting down
2026/08/10 02:21:02 [notice] 36#36: gracefully shutting down
2026/08/10 02:21:02 [notice] 34#34: exit
2026/08/10 02:21:02 [notice] 35#35: exiting
```



Muestra los mensajes y eventos que ha generado el contenedor. Sirve para ver errores o comprobar que todo funciona bien.

docker exec



Permite entrar a un contenedor o ejecutar comandos dentro de él mientras está funcionando. Diferencia clave con `docker run`: run crea un contenedor nuevo, exec entra a uno que ya existe y está prendido.

docker inspect

```
casti@MLC MINGW64 ~/taller5_docker
$ docker inspect determined_noether
[
    {
        "Id": "e18ed83bf41a9c93ea52f345fe2f88b22cc8e02a71d66652c163068c5f73e18f",
        "Created": "2026-08-10T02:15:26.374953099Z",
        "Path": "/docker-entrypoint.sh",
        "Args": [
            "nginx",
            "-g",
            "daemon off;"
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 705,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-08-10T02:23:11.813730437Z",
            "FinishedAt": "2026-08-10T02:21:02.683253605Z"
        },
        "Image": "sha256:8541484afbc9c8a5a8a99b379568ebbc957f658583ec9448fc43104229c03cf8",
        "ResolvConfPath": "/var/lib/docker/containers/e18ed83bf41a9c93ea52f345fe2f88b22cc8e02a71d66652c163068c5f73e18f/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/e18ed83bf41a9c93ea52f345fe2f88b22cc8e02a71d66652c163068c5f73e18f/hostname",
        "HostsPath": "/var/lib/docker/containers/e18ed83bf41a9c93ea52f345fe2f88b22cc8e02a71d66652c163068c5f73e18f/hosts",
        "LogPath": "/var/lib/docker/containers/e18ed83bf41a9c93ea52f345fe2f88b22cc8e02a71d66652c163068c5f73e18f/e18ed83bf41a9c93ea52f345fe2f88b22cc8e02a71d66652c163068c5f73e18f-json.log",
        "Name": "/determined_noether",
        "RestartCount": 0,
        "Driver": "overlayfs",
        "Platform": "linux",
        "MountLabel": "",
        "ProcessLabel": "",
        "AppArmorProfile": "",
        "ExecIDs": null,
        "HostConfig": {
            "Binds": null,
            "ContainerIDFile": "",
            "LogConfig": {
                "Type": "json-file",
                "Config": {}
            },
            "NetworkMode": "bridge",
            "PortBindings": {
                "80/tcp": [
                    {
                        "HostIp": "",
                        "HostPort": "8080"
                    }
                ]
            },
            "RestartPolicy": {
                "Name": "no",
```

```
        },
        "Networks": {
            "bridge": {
                "IPAMConfig": null,
                "Links": null,
                "Aliases": null,
                "DriverOpts": null,
                "GwPriority": 0,
                "NetworkID": "005af8ecdcfb3220874a41bab2aac97f3a7dbb566bb34bf4e77ecb1f5d516ff6",
                "EndpointID": "7cd1414894a355e805b66f13e4048a84bcaadf58070a6ed65b4407bf5dc0be6d",
                "Gateway": "172.17.0.1",
                "IPAddress": "172.17.0.2",
                "MacAddress": "6a:18:f6:18:22:40",
                "IPPrefixLen": 16,
                "IPv6Gateway": "",
                "GlobalIPv6Address": "",
                "GlobalIPv6PrefixLen": 0,
                "DNSNames": null
            }
        }
    },
    "ImageManifestDescriptor": {
        "mediaType": "application/vnd.oci.image.manifest.v1+json",
        "digest": "sha256:963cfe6e75d1c292f66589d7e190b137cf89310414c0c1c5b476dfc61a4fcd0d",
        "size": 2290,
        "annotations": {
            "com.docker.official-images.bashbrew.arch": "amd64",
            "org.opencontainers.image.base.digest": "sha256:38a76d01668772e381ad2826d876627c89e7133e2f8a0f5d567306798b0f2a16",
            "org.opencontainers.image.base.name": "debian:trixie-slim",
            "org.opencontainers.image.created": "2026-08-05T00:21:36Z",
            "org.opencontainers.image.revision": "ccdab6c99ae2e2fc53a144dc68d6b8f44163adf2",
            "org.opencontainers.image.source": "https://github.com/nginx/docker-nginx.git#ccdab6c99ae2e2fc53a144dc68d6b8f44163adf2:mainline/debian",
            "org.opencontainers.image.url": "https://hub.docker.com/_/nginx",
            "org.opencontainers.image.version": "1.31.3"
        },
        "platform": {
            "architecture": "amd64",
            "os": "linux"
        }
    }
```

Muestra información detallada del contenedor, como configuración, red, puertos e identificadores.

docker rm

```
casti@MLC MINGW64 ~/taller5_docker
$ docker rm determined_noether
Error response from daemon: cannot remove container "determined_noether": container is running: stop the container before removing or force remove

casti@MLC MINGW64 ~/taller5_docker
$ docker stop determined_noether
determined_noether

casti@MLC MINGW64 ~/taller5_docker
$ docker rm determined_noether
determined_noether
```

Elimina el contenedor que le ponga, que ya no necesite.

docker rmi

```
casti@MLC MINGW64 ~/taller5_docker
$ docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:8541484afbc9c8a5a8a99b379568ebbc957f658583ec9448fc43104229c03cf8
```

Elimina la imagen de Docker que ya no se va a utilizar.

Parte 7: Recursos Docker

```
casti@MLC MINGW64 ~/taller5_docker
$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
005af8ecdcfb   bridge    bridge    local
bd71e45283c3   host      host      local
0338cf9aab6d   none      null      local

casti@MLC MINGW64 ~/taller5_docker
$ docker volume ls
DRIVER    VOLUME NAME

casti@MLC MINGW64 ~/taller5_docker
$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          4         4         4.372GB   4.372GB (100%)
Containers      10        0         274.4kB   274.4kB (100%)
Local Volumes   0         0         0B        0B
Build Cache     0         0         0B        0B

casti@MLC MINGW64 ~/taller5_docker
$ docker system prune
WARNING! This will remove:
  - all stopped containers
  - all networks not used by at least one container
  - all dangling images
  - unused build cache

Are you sure you want to continue? [y/N] N
```

Reto integrador

1. Contenedor Ubuntu

```
casti@MLC MINGW64 ~/taller5_docker
$ docker run -dit --name mi_ubunticoo ubuntu bash
ca51d5ac812b756db5b93b254e0efe724598025d534ce3751388eed65d782b64
```

`docker run -dit --name mi_ubuntu ubuntu bash` crea un contenedor basado en la imagen de Ubuntu y lo deja ejecutándose en segundo plano gracias a `-d` (el programa sigue ejecutándose, pero sin ocupar la terminal). Así el contenedor está encendido, pero no estoy metida dentro de él, y puedo acceder más tarde. `--name mi_ubuntu` le asigna un nombre fácil de recordar en vez del nombre aleatorio que Docker genera.

2. Contenedor Nginx

```
casti@MLC MINGW64 ~/taller5_docker
$ docker run -d -p 8080:80  --name mi_nginx nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
5a4222b844e8: Pull complete
d84ae7b21412: Pull complete
b8b80b9bc028: Pull complete
26c307b5e35a: Pull complete
c0df8d325117: Pull complete
f5de6e85ac74: Pull complete
3c55dc422a81: Pull complete
0f03cb4db0ef: Download complete
92fcf0fc2ef2: Download complete
Digest: sha256:8541484afbc9c8a5a8a99b379568ebbc957f658583ec9448fc43104229c03cf8
Status: Downloaded newer image for nginx:latest
677990877d9b1b8965899597553c60291800ca5b9e19ddc516f2535efc869240
```

`docker run -d -p 8080:80 --name mi_nginx nginx` crea y ejecuta un contenedor de Nginx. `-p 8080:80` conecta el puerto 8080 de mi computador con el puerto 80 del contenedor, permitiendo acceder al servidor web desde el navegador con `localhost:8080`.

3. Verificar que ambos están corriendo

```
casti@MLC MINGW64 ~/taller5_docker
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                     NAMES
677990877d9b   nginx     "/docker-entrypoint.…"   2 minutes ago   Up 2 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   mi_nginx
ca51d5ac812b   ubuntu    "bash"                   8 minutes ago   Up 8 minutes                                             mi_ubunticoo
```

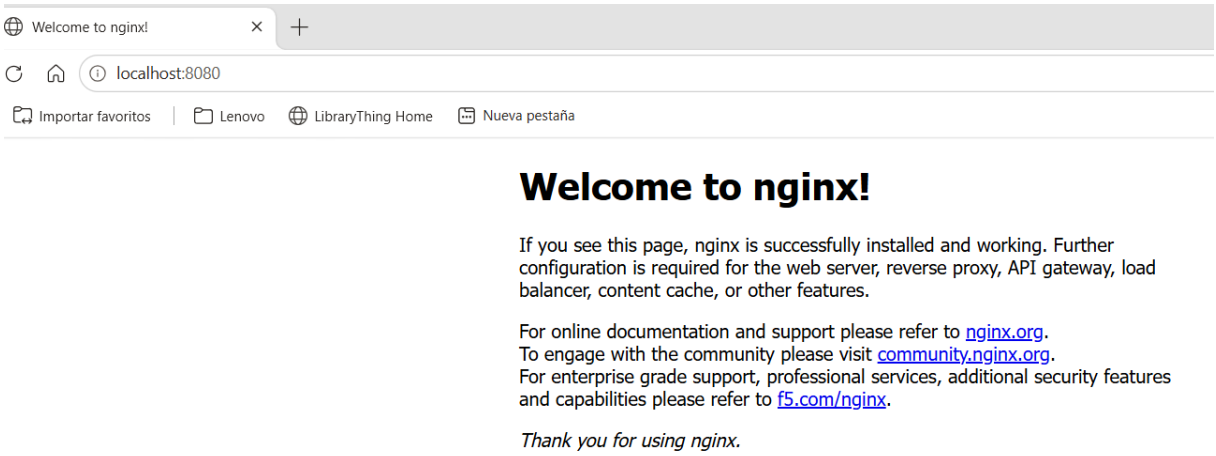
`docker ps` muestra los dos contenedores en ejecución.

4. Acceder a Ubuntu con docker exec

```
casti@MLC MINGW64 ~/taller5_docker
$ docker exec -it mi_ubunticoo bash
root@ca51d5ac812b:/# whoami
root
root@ca51d5ac812b:/# pwd
/
root@ca51d5ac812b:/# exit
exit
```

El comando sirve para meterme dentro del contenedor de Ubuntu que está en ejecución. Probé `whoami` para ver el usuario, después `pwd`, que mostró la ruta actual `/`, y finalmente `exit` para salir y regresar a la terminal de mi computador.

5. Logs de Nginx




```
casti@MLC MINGW64 ~/taller5_docker
$ docker logs mi_nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/10 03:01:58 [notice] 1#1: using the "epoll" event method
2026/08/10 03:01:58 [notice] 1#1: nginx/1.31.3
2026/08/10 03:01:58 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/08/10 03:01:58 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/08/10 03:01:58 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/08/10 03:01:58 [notice] 1#1: start worker processes
2026/08/10 03:01:58 [notice] 1#1: start worker process 29
2026/08/10 03:01:58 [notice] 1#1: start worker process 30
2026/08/10 03:01:58 [notice] 1#1: start worker process 31
2026/08/10 03:01:58 [notice] 1#1: start worker process 32
2026/08/10 03:01:58 [notice] 1#1: start worker process 33
2026/08/10 03:01:58 [notice] 1#1: start worker process 34
2026/08/10 03:01:58 [notice] 1#1: start worker process 35
2026/08/10 03:01:58 [notice] 1#1: start worker process 36
2026/08/10 03:01:58 [notice] 1#1: start worker process 37
2026/08/10 03:01:58 [notice] 1#1: start worker process 38
2026/08/10 03:01:58 [notice] 1#1: start worker process 39
2026/08/10 03:01:58 [notice] 1#1: start worker process 40
```

Permite ver los registros del contenedor Nginx. Se observa que Docker ejecutó varios scripts de configuración inicial, habilitó IPv6 y cargó correctamente la configuración del servidor. También muestra la versión de Nginx y el sistema operativo donde corre.

6. Detener ambos contenedores

```
casti@MLC MINGW64 ~/taller5_docker
$ docker stop mi_ubunticoo mi_nginx
mi_ubunticoo
mi_nginx
```

Se usó para apagar los dos contenedores de forma controlada antes de eliminarlos.

7. Eliminar los contenedores

```
casti@MLC MINGW64 ~/taller5_docker
$ docker rm mi_ubunticoo mi_nginx
mi_ubunticoo
mi_nginx
```

Elimina los contenedores detenidos para liberar recursos.

8. Eliminar las imágenes

```
casti@MLC MINGW64 ~/taller5_docker
$ docker rmi ubuntu nginx
Untagged: ubuntu:latest
Deleted: sha256:678c6550cc43645e08669028bc177f50be4e7c5b8cca677067b1914d4afc7a03
```

Elimina las imágenes de Ubuntu y Nginx para liberar espacio en disco.